

EPITOUS – COMPLETE LEARNING & WORK SYSTEM

From Zero to Autonomous Developer

EpiTous accompagne un débutant complet dans sa journée de travail, lui apprend les fondamentaux, lui montre comment apprendre, chercher, pratiquer, debugger et résoudre des problèmes, puis l'amène progressivement vers l'autonomie.

Learn

Practice

Research

Debug

Reflect

Progress

Autonomy

kajarnaklokossou2008.github.io/epitous/

EpiTous n'est pas un guide. C'est une structure complète de travail et d'apprentissage. L'objectif ultime n'est pas de faire réussir l'étudiant grâce à EpiTous. C'est de lui apprendre à réussir sans EpiTous.

EpiTous est une plateforme pédagogique interactive qui accompagne un débutant complet dans son apprentissage de la programmation et dans son travail quotidien. Elle combine cours progressifs, vidéos, documentation, recherche guidée, exercices, environnement de pratique, debugging, journal d'apprentissage, suivi de progression et assistance intelligente.

Son premier parcours est conçu autour de la Piscine Epitech : l'étudiant commence sans prérequis, apprend les fondamentaux de l'informatique, Unix et C, découvre ensuite les contraintes spécifiques de son environnement pédagogique, puis applique progressivement ses connaissances aux tâches de la Piscine.



La courbe d'autonomie

L'assistance diminue progressivement jusqu'à l'autonomie complète :

- **Débutant** : "Voici quoi faire."
- **Intermédiaire** : "Voici les ressources. Choisis."
- **Avancé** : "Voici ton objectif. Organise ton travail."
- **Autonome** : "Voici le problème. Résous-le."



Jour 1 — L'ordinateur et toi

- | | | |
|-------|--|--|
| 09:00 | | Warm-up : Qu'est-ce qu'un ordinateur ? Qu'est-ce qu'un fichier ? |
| 09:15 | | Cours : OS, terminal, clavier, commandes, chemins, programme |
| 09:45 | | Recherche : Lire "Qu'est-ce que la programmation" |
| 10:15 | | Vidéo : How computers work |
| 10:45 | | Pratique : ls, cd, pwd, mkdir, touch dans le terminal |
| 11:30 | | Epitech Mode : Organisation de l'environnement Piscine |
| 14:00 | | Tâches : Configurer ton espace de travail |
| 17:45 | | Revue : Qu'as-tu appris aujourd'hui ? |

Jour 2 — Le terminal : ton meilleur ami

- 09:00  Warm-up : Révision du Jour 1
- 09:15  Cours : ls, cd, pwd, mkdir, rm, cp, mv, cat, echo, man
- 09:45  Recherche : Lire le manuel de ls (man ls)
- 10:45  Pratique : Naviguer le système de fichiers, créer/supprimer
- 11:30  Epitech Mode : La norme dans l'usage du terminal

Jour 3 — Premier programme C

- 09:00  Warm-up : Révision des commandes terminal
- 09:15  Cours : Qu'est-ce que C, hello world, compilation, exécution
- 10:15  Vidéo : C programming for beginners
- 10:45  Pratique : Écrire et compiler hello world
- 11:30  Epitech Mode : Coding style, Makefile basique

```
// Hello World en C
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

Jour 4 — Variables et types

- 09:00  Warm-up : Révision de la compilation
- 09:15  Cours : int, char, float, double, déclaration, printf
- 10:45  Pratique : Déclarer des variables, les afficher

```
int age = 25;
char initial = 'K';
float taille = 1.75;

printf("Age: %d, Initial: %c, Taille: %.2f\n", age, initial, taille);
```

Jour 5 — Fonctions

09:00  Warm-up : Révision des variables

09:15  Cours : Signature, paramètres, valeur de retour, void

10:45  Pratique : Écrire une fonction qui additionne deux nombres

```
int add(int a, int b)
{
    return a + b;
}

// Utilisation
int result = add(3, 7); // result = 10
```

Jour 6 — Conditions et boucles

09:00  Warm-up : Révision des fonctions

09:15  Cours : if/else, while, for, break, continue

10:45  Pratique : Boucle qui affiche les nombres 1 à 10

```
for (int i = 1; i ≤ 10; i++) {
    printf("%d\n", i);
}

if (age ≥ 18) {
    printf("Majeur\n");
} else {
    printf("Mineur\n");
}
```

Jour 7 — Pointeurs (le jour critique)

09:00  Warm-up : Révision de tout ce qui précède

09:15  Cours : Qu'est-ce qu'un pointeur, & (adresse), * (déréférencement)

10:15  Vidéo : Pointers explained

10:45  Pratique : Déclarer un pointeur, obtenir une adresse, déréférencer

```
int x = 42;
int *ptr = &x; // ptr pointe vers l'adresse de x

printf("Valeur de x: %d\n", x);           // 42
printf("Adresse de x: %p\n", (void*)&x);
printf("Valeur via ptr: %d\n", *ptr);    // 42
```

```
*ptr = 100; // Modifie x via le pointeur
printf("Nouvelle valeur de x: %d\n", x); // 100
```

03 Glossaire essentiel

C

Unix

Git

Général

Variable

Une variable est un espace de mémoire qui stocke une valeur. En C, on doit déclarer le type : `int age = 25;`

C

Pointeur

Un pointeur est une variable qui stocke l'adresse mémoire d'une autre variable. `int *ptr = &x;` — `ptr` contient l'adresse de `x`. On déréférence avec `*ptr` pour accéder à la valeur.

C

Fonction

Bloc de code réutilisable qui prend des paramètres en entrée et retourne une valeur. `int add(int a, int b) { return a + b; }`

Unix

Terminal

Interface texte qui permet d'interagir avec le système d'exploitation via des commandes. Exemples : `ls`, `cd`, `mkdir`.

Unix

Compilation

Processus qui transforme le code source (C) en un exécutable. `gcc main.c -o program` produit un fichier `program` qu'on peut exécuter.

C

Makefile

Fichier qui automatise la compilation. Contient des règles qui décrivent comment compiler le projet. `make` exécute la première règle.

Git

Commit

Un instantané (snapshot) de ton code à un moment donné. `git commit -m "message"` sauvegarde les modifications avec un message descriptif.

Git

Branch (Branche)

Une ligne de développement parallèle. `git branch feature` crée une branche nommée "feature" pour travailler sans impacter la branche principale.

Général

Fichier vs Dossier

Un fichier stocke des données (code, image, texte). Un dossier (directory) contient des fichiers et d'autres dossiers, organisant la structure.

Unix

Chemin (Path)

Localisation d'un fichier ou dossier dans l'arborescence. Absolu : `/home/user/projet`. Relatif : `../projet`.

04 Error Lab — Erreurs courantes

```
segmentation fault (core dumped)
```

Erreur de segmentation

Cause : Ton programme accède à une zone de mémoire interdite — souvent un pointeur non initialisé, un accès hors limites d'un tableau, ou un déréférencement de pointeur NULL.

Debug :

- Vérifie tous tes pointeurs — sont-ils initialisés ?
- Vérifie les bornes de tes tableaux (index 0 à n-1, pas n)
- Utilise `valgrind ./program` pour trouver la ligne exacte
- Vérifie que tu ne déréférences pas un pointeur NULL

```
undefined reference to `main'
```

Référence non définie à main

Cause : Le compilateur ne trouve pas la fonction `main`. Soit elle n'existe pas, soit elle est mal orthographiée, soit tu compiles le mauvais fichier.

Debug : Vérifie que tu as `int main(void)` ou `int main(int argc, char **argv)` dans ton fichier C.

```
implicit declaration of function 'printf'
```

Déclaration implicite de fonction

Cause : Tu utilises une fonction sans avoir inclus le header qui la déclare. Pour `printf`, il faut `#include <stdio.h>`.

Debug : Ajoute le `#include` approprié en haut de ton fichier.

```
make: *** No rule to make target
```

Pas de règle dans le Makefile

Cause : Le Makefile ne contient pas de règle pour la cible demandée, ou il y a une faute de frappe dans le nom de la cible.

Debug : Vérifie ton Makefile — les tabulations (pas d'espaces !) et les noms de cibles.

05 Roadmap

V1 — Frontend statique (actuel)

Landing page, Dashboard, 7 jours de curriculum, Journal, Glossaire, Error Lab, Paths. Persistance localStorage. Disponible sur GitHub Pages.

V2 — Backend + comptes utilisateurs

Sauvegarde de progression côté serveur, authentification, synchronisation multi-appareils.

V3 — Code runner sécurisé

Exécution de code C dans un environnement sandbox (Docker). L'étudiant peut tester son code directement dans EpiTous.

V4 — EpiTutor (IA)

Assistant IA qui analyse le code de l'étudiant, donne du feedback personnalisé et suggère des exercices adaptés.

V5 — Forum / Communauté

Les étudiants peuvent s'entraider, partager leurs solutions (après validation), et poser des questions.

V6 — Personnalisation + Analytics

Parcours personnalisés selon le rythme et les difficultés de chacun. Analytics pour suivre la progression globale.

L'objectif ultime

EpiTous n'est pas un guide. C'est une structure complète de travail et d'apprentissage. L'objectif n'est pas de faire réussir l'étudiant grâce à EpiTous. C'est de lui apprendre à réussir sans EpiTous.

kajarnaklokossou2008.github.io/epitous/